Advanced Attacks in Assembly x64 (Canary and GOT)

NECKER

November 17, 2025

Contents

1 Analysis of Classic Buffer Overflow Examples (Stack 1, 4, 5)

These examples illustrate how a Buffer Overflow is exploited to manipulate local variables (the "cookie") instead of the return address, for educational purposes. The vulnerability in all cases is the use of the `gets()` function.

1.1 1. Stack 1.c: Simple and Fixed Objective

`stack1.c` demonstrates the basic concept of a Buffer Overflow to alter a local variable.

```c
int main()
{
        int cookie;
        char buf[80];

        printf("buf: %08x cookie: %08x\n", &buf, &cookie);
        gets(buf);

        if (cookie == 0x41424344)
        printf("you win!\n");
}
```

- **Vulnerability:** The `gets(buf)` function reads input without bounds, allowing data to be written beyond the 80 bytes of `buf`.
- **Objective:** The `cookie` variable is allocated on the Stack immediately after `buf`.
- **Attack:** The attacker provides a string of 80 bytes + 4 bytes (padding/alignment) followed by the target value `0x41424344` (which corresponds to the string "ABCD" in ASCII, endianness permitting) to overwrite the `cookie` and win.
  WARNING: they must be written in reverse because integers are in little endian.
- **Debug Output:** The use of `%08x` suggests a 32-bit architecture (x86), where addresses and integers are 4 bytes long.

1.2 2. Stack 4.c: Attacking the Return Address (The Null Byte Problem)

While the previous code aimed to overwrite the `cookie` variable, it becomes difficult here because the `cookie` contains terminating characters; to break `stack4.c`, we must instead aim to overwrite the return address (**RET**).

```
int main()
{
        int cookie;
        char buf[80];

        printf("buf: %08x cookie: %08x\n", &buf, &cookie);
        gets(buf);

        if (cookie == 0x000d0a00)
        printf("you win!\n");
}
```

The Objective of the Attack (Overwriting the RET):

- **Target:** The return address (**RET**) located in the Stack Frame, after `buf` and `cookie`.
- **Action:** The attacker must send a string of 80 bytes + Padding (for local variables) followed by the **new Return Address** (the address of the shellcode).

The Crucial Challenge: The `gets()` Function

The `gets()` function reads input until it encounters a newline character (`\n`) or, more critically, a **Null Byte** (`\0`).

- **Potential Problem:** The address of the **shellcode** or a useful function (like the address of `system()`) almost always contains one or more null bytes (`0x00`) at the beginning or end.
- **The Failure:** If the attacker tries to include an address containing `0x00` inside the payload for `gets()`, the `gets()` function **will terminate reading prematurely** upon encountering the first null byte. The return address will not be entirely overwritten, and the attack will fail.
- **The Solution:** To exploit `gets()`, the attacker must find a way to inject *binary* input that includes the null byte, often by sending the payload via a file or a pipe, and must ensure that the chosen jump address **does not contain null bytes** (*null-byte free*) or that the null byte is strategically positioned to terminate the string after the address and not within it.

1.3 3. Stack 5.c: Transition to x64 (64-bit)

`stack5.c` introduces the context of a 64-bit architecture, which further complicates the alignment and size of variables.

```
int main()
{
        int cookie;
        char buf[80];

        printf("buf: %16lx cookie: %16lx\n", &buf, &cookie);
        gets(buf);

        if (cookie == 0x000d0a00)
        printf("you loose!!\n");
}
```

- **Debug Output:** The use of `%16lx` suggests an **x64 (64-bit)** architecture, where pointers are 8 bytes long (16 hexadecimal digits).
- **Control Variable:** Despite the 64-bit architecture, the `cookie` variable remains an `int` type (usually 4 bytes).
- **Implication of the Overflow:** The space between `buf[80]` and the `cookie` variable is now more complex and likely includes **padding** (empty space) to ensure that the address of `cookie` is correctly aligned to 4 or 8 bytes (as required by x64 conventions).

- **Attack:** The attacker must precisely calculate the length of the padding (which can be up to 4 extra bytes) beyond the 80 bytes of `buf` to correctly hit `cookie`.
  WARNING: however, if we guess the cookie now, we lose, so we inject shellcode instead:

1.3.1 3. Stack 5.c: Shellcode Injection and Return-to-Stack

It is possible to execute a classic Buffer Overflow attack (if modern defenses are disabled) by injecting malicious code (*shellcode*) directly into the Stack and forcing the program to execute it.

Prerequisites for the Attack:

- **Executable Stack (NX bit disabled):** The section of the Stack where the *shellcode* is injected must allow code execution.
- **Canary Disabled:** Stack Canary protection must be absent or ineffective to allow overwriting the RET.

Phases of the Attack:

1. **Calculating the Injection Address:** The attacker uses the output of `printf` (`%16lx`) to determine the exact address of `buf` (e.g., ADDR_{buf}).
2. **Constructing the Payload:** The payload (the string provided to `gets()`) is structured as follows:

$$\text{Payload} = \underbrace{\text{Shellcode}}_{\text{In binary}} \;||\; \underbrace{\text{Padding}}_{\text{Filling up to the RET}} \;||\; \underbrace{\text{ADDR}_{buf}}_{\text{New Return Address (we must know it exactly)}}$$

3. **Injection:** The *shellcode* is injected into the initial part of the buffer `buf`. The length of the shellcode and padding fills the entire space between the beginning of `buf` and the **Return Address (RET)**.
4. **Hijacking the RET:** The last 8 bytes of the payload (on x64) overwrite the RET with the address of `buf` (ADDR_{buf}).
5. **Execution:** When the `main` function finishes, instead of executing the `RET` instruction to return to the legitimate caller, the processor jumps to the injected address, which is the beginning of `buf` where the **shellcode** resides. The shellcode is then successfully executed.

2 Main Defense: The Stack Canary

Modern attacks exploit the logical separation of memory to bypass defenses.

Element	Where it is Located	Purpose
Return Address (RET)	Stack Frame	Return address of the function.
Stack Canary	Stack Frame	Protection of the RET.
Global Offset Table (GOT)	Data/BSS Section	Contains the addresses of library functions (`exit`, `printf`).

Table 1: Differences between Stack and Data

- **Function:** The Canary is a secret value placed in the Stack Frame between local variables (e.g., the buffer `buf`) and the Return Address (RET).
- **Protection Mechanism:** Before the function terminates, the program verifies the integrity of the Canary.
- **Reaction to Failure:** If the Canary has been modified (a sign of a Buffer Overflow), the program **does not** execute the `RET` instruction. Instead, it jumps to an error handling function that executes `exit()` to terminate the program safely.

3 Advanced Attack: Bypassing the Canary with Return-to-GOT (Abo5.c)

```
int main(int argv, char **argc) {
        char *pbuf = malloc(strlen(argc[2]) + 1);
        char buf[256];

        strcpy(buf, argc[1]);

        for (; *pbuf++ = *(argc[2]++); );
        exit(1);
}
```

The attack focuses on bypassing the Canary defense, targeting the **GOT** when the Canary forces the program to exit.

3.1 The Target: The Global Offset Table (GOT)

- The **GOT** is the list of pointers that the program uses to call functions in external libraries (such as `exit`).
- **The Attack:** Overwriting the address of the `exit` function inside the GOT with the address of the attacker's shellcode.

3.2 Mechanism of the Exploit in 3 Phases

Phase 1: Obtaining Arbitrary Write (Pointer Hijacking)

- **Vulnerability:** `strcpy(buf, argv[1])`
- **Action:** The attacker provides a string in `argv[1]` that is long enough to overflow `buf[256]` and overwrite the local pointer `char *pbuf` on the Stack.
- **Result:** `pbuf` is forced to point to the address of the **GOT entry of** `exit` (WARNING: the address of the GOT must be known).

Phase 2: Writing the Payload (Shellcode)

- **Vulnerability:** `for (; *pbuf++ = *argc[2]; );`
- **Action:** The attacker uses `argv[2]` to provide the **shellcode code** or the address of the target attack function. The loop copies this content into the address pointed to by `pbuf`.
- **Result:** The **GOT entry of** `exit` is overwritten with the address of the shellcode.

Phase 3: Executing Malicious Code

- **Trigger:** The program reaches the line `exit(1);` (or the Canary check fails and calls `exit()`).
- **Execution:** The `CALL exit` instruction consults the **GOT**. It reads the shellcode address written by the attacker.
- **Conclusion:** The execution flow jumps to the shellcode, completely bypassing the Stack Canary protection.

4 Advanced Attacks in Protected Environments (NX bit)

The previous techniques (shellcode injection into the Stack) fail if the Stack is marked as **non-executable** (thanks to the *No-eXecute bit*, or NX bit). The next example shows how to bypass this restriction. WARNING: in the next example, the canary must not be active.

4.1 RET2libc / ROP CHAIN: Executing Existing Code

If the Stack is non-executable, the attacker cannot execute injected code. The solution is to **reuse** code that is already in memory and guaranteed to be executable, such as standard library functions in `libc` (hence the name RET2libc, Return-to-libc). It is called a chain because it uses existing but scattered code and links it together sequentially to perform our required actions.

```
int gadget()
{
        asm(
        "pop %rdi\n"
        "ret");
}

int main()
{
        int cookie;
        char buf[80];

        printf("puts(): %16lx\n", &puts);
        printf("gadget(): %16lx\n", &gadget);
        printf("buf: %16lx cookie: %16lx\n", &buf, &cookie);
        gets(buf);

        if (cookie == 0x000d0a00)
        printf("you loose!\n");
}
```

- **Objective:** Call an existing function (a *gadget* or a library function, e.g., `system()`) by using the overflow to overwrite the **Return Address (RET)**.
- **The x64 Problem:** To call a function (e.g., `puts()`), the x64 architecture (System V ABI) requires that the first argument (e.g., the address of the string to print) be loaded into the `RDI` **register**. The Stack Overflow allows control only over the RET, not the registers.
- **Solution: POP RDI Gadget (ROP Chain):** The attacker looks through the program's memory for a short sequence of Assembly instructions called a **Gadget** (e.g., `"pop rdi; ret"`). This gadget, when executed:
  1. Extracts the next value from the Stack and loads it into the `RDI` register.
  2. Executes `RET`, which jumps to the subsequent address on the Stack.

Structure of the ROP Chain (Hijacked Stack):
The execution chain is constructed entirely within the non-executable Stack:

1. The original **RET** is overwritten with the address of the **Gadget** (`pop rdi; ret`).
2. Immediately after, the **Argument** for the function is placed (e.g., the pointer to the string `"You Win!\0"`).
3. Immediately after that, the **Address** of the function `puts()` (or `system()`) is placed.

When the program executes the RET, it runs the Gadget, which prepares `RDI` and jumps to the function, achieving execution with the desired parameters.

4.2 Note

Let us remember that the **Stack** and the **.text** section (the section where the code is written) are unaware of each other and rely on being properly aligned in what they do; this example acts to misalign the **Stack** so that it fetches what we want as if it were the **RET**.

THE STACK AFTER THE OVERFLOW OVERWRITE:

Relative Address	Content (Injected Payload)
Old RBP	Address of the previous `main` (8 bytes)
Old RET	**&gadget()**
RET + 8	**String Address** (`&"You Win!\0"`)
RET + 16	**&puts()**
buf	80 bytes to fill - the ones used
\ non-executable stack	
\ non-executable stack	
String	`&"You Win!\0"`

5 Practical Example: PwnCollege Memory Errors – Level 1.1

This example illustrates an exploit that does not target the control flow (*RET* or *GOT*), but rather the program **logic** by overwriting a control variable.

```
; [RBP - 0x90] contains the address of the buffer (RBP-0x80)
; we know this from these 2 preceding instructions:
lea rax, [rbp - 0x80]
mov QWORD PTR [rbp - 0x90], rax ; saves 0x80 into 0x90

; Loads the address of the buffer into RAX
mov rax, QWORD PTR [rbp-0x90]

; Sets EDI (RDI) = 0 (tells read to take from stdin
; (it will later place it into what is written in rsi))
mov edi, 0x0

; Sets RSI (Argument 2) = Address of the buffer (RBP-0x80)
mov rsi, rax

; Loads the size of the read into RDX (Argument 3)
; The value here is excessive, causing the overflow
mov rdx, QWORD PTR [rbp-0x98]

; Executes the read function. The overflow occurs here.
call 0x11a0 <read@plt>
```

Challenge Objective:

- Find a way to call the `win()` method.

Victory Mechanism:

- The disassembled code shows that the program performs a check: if the value pointed to by the local variable `[RBP - 0x88]` is non-zero (`!= 0`), the `win()` function is called.

The Exploitable Vulnerability (Indirect Arbitrary Write):

- The `read()` function (which is unsafe, similar to `gets`) reads data into a buffer whose starting position is stored in `[RBP - 0x90]` (which, in turn, contains the address RBP − 0x80).
- The overflow occurs when the input to `read()` exceeds the buffer size.
- **Attack Strategy:** The attacker exploits the overflow to:
  1. **Overwrite the Pointer:** The `read()` function is called to write into the buffer. The first overflow is large enough to overwrite the address stored in `[RBP - 0x88]` (the pointer to the control variable).

2. **Overwrite the Value:** Input is provided such that, in a subsequent step, a non-zero value (*e.g.*, `1`) is written to the address that `[RBP - 0x88]` now points to.

- **Conclusion:** Since the user can control the size of the input, it is possible to overwrite the value verified by the `if` condition and force the execution of the `win()` function.